

Guia de reciclaje de componentes web

Funcionalidades 26 a 40 - continuacion del recurso unificado

HTML + CSS + JavaScript modular y version normal

Este documento mantiene el mismo enfoque de la guia anterior: cada funcionalidad se explica como una pieza reciclable. La idea es que puedas copiar el bloque HTML, el bloque CSS y el bloque JavaScript, cambiar los ids, clases, data-attributes o variables necesarias, y reutilizarlo en cualquier proyecto web.

El orden sigue la tabla de contenido del recurso unificado: desde la funcionalidad 26, Modal, hasta la 40, Exportar JSON.

Rango	Funcionalidades cubiertas	Objetivo de esta guia
26 a 40	Modal, Toast, Carrusel, Tooltip, Badge, Copy clipboard, Cards dinamicas, Busqueda debounce, Filtros chips, Ordenamiento, Paginacion, Tabla ordenable, Filtro de tabla, Notas localStorage y Exportar JSON.	Aprender a reciclar cada componente en otros proyectos con instrucciones claras, variables a cambiar y ejemplos en codigo modular y normal.

Tabla de contenido

- 26. Modal reusable
- 27. Toast / notificaciones flotantes
- 28. Carrusel / slider
- 29. Tooltip / ayuda contextual
- 30. Badge / indicador numerico
- 31. Copy clipboard / copiar al portapapeles
- 32. Cards dinamicas desde un array
- 33. Búsqueda con debounce
- 34. Filtros chips por categoria
- 35. Ordenamiento de resultados
- 36. Paginacion
- 37. Tabla ordenable por columnas
- 38. Filtro de tabla por texto
- 39. Notas con localStorage
- 40. Exportar JSON

Como usar esta guia

- 1 Elige la funcionalidad que necesitas reutilizar.
- 2 Copia primero el HTML minimo del componente.
- 3 Copia el CSS asociado y modifica colores, tamaños o variables visuales.
- 4 Escoge una sola version de JavaScript: modular con export/import o normal sin modulos.
- 5 Cambia ids, clases y data-attributes segun la tabla de variables de cada funcionalidad.
- 6 Prueba la funcionalidad con el checklist antes de integrarla con otros componentes.

Regla importante: no mezcles la version modular y la version normal al mismo tiempo. Si usas import/export, carga el HTML con type="module". Si usas JS normal, usa un script comun sin import/export.

Convenciones que se repiten en las funcionalidades 26 a 40

Convencion	Para que sirve	Como adaptarla
data-*	Guarda informacion en HTML para que JavaScript la lea sin depender del texto visible.	Cambia valores como data-copy, data-category, data-page o data-open-modal segun tu proyecto.
is-active / is-open / is-hidden	Clases de estado agregadas por JavaScript para cambiar la vista.	No las escribas manualmente salvo estado inicial; JS debe agregarlas o quitarlas.
containerId	Id del contenedor donde se renderiza o controla el componente.	Debe coincidir con el id del HTML.
callback / onResults / onPageChange	Funcion que se ejecuta cuando un componente produce resultados.	Usala para conectar filtros, busqueda, paginacion o render de cards.
localStorage key	Nombre con el que guardas datos en el navegador.	Cambia claves como kit-notes si vas a mezclar varios proyectos.

Ejemplo de conexion entre componentes: Cards dinamicas + Busqueda debounce + Filtros chips + Ordenamiento + Paginacion. En proyectos reales, lo mas limpio es tener un estado global con los datos originales, filtros activos, texto de busqueda, orden elegido y pagina actual.

26. Modal reutilizable

Aspecto	Explicacion
Que hace	Ventana emergente para confirmar acciones, mostrar formularios, avisos, detalles o contenido que no debe ocupar toda la pagina.
Donde se usa en industria	Confirmar eliminaciones, login rapido, politicas, vista previa de producto, formulario de contacto, mensajes de exito o errores importantes.
Como se personaliza	Cambia el id del modal, los botones data-open-modal y data-close-modal, el titulo, el contenido y el comportamiento de cierre.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<button class="btn" data-open-modal="modalContacto">Abrir modal</button>

<div class="modal" id="modalContacto" aria-hidden="true">
  <div class="modal__overlay" data-close-modal></div>

  <article class="modal__content" role="dialog" aria-modal="true" aria-labelledby="modalTitulo">
    <button class="modal__close" type="button" data-close-modal aria-label="Cerrar modal">x</button>
    <h2 id="modalTitulo">Contactar asesor</h2>
    <p>Completa tus datos y nos comunicaremos contigo.</p>
    <form class="form">
      <input type="text" placeholder="Nombre" required>
      <input type="email" placeholder="Correo" required>
      <button class="btn" type="submit">Enviar</button>
    </form>
  </article>
</div>
```

CSS del componente

styles.css

```

.modal {
  position: fixed;
  inset: 0;
  display: grid;
  place-items: center;
  padding: 24px;
  opacity: 0;
  visibility: hidden;
  pointer-events: none;
  z-index: 1000;
  transition: opacity .25s ease, visibility .25s ease;
}
.modal.is-open {
  opacity: 1;
  visibility: visible;
  pointer-events: auto;
}
.modal__overlay {
  position: absolute;
  inset: 0;
  background: rgba(15, 23, 42, .65);
  backdrop-filter: blur(10px);
}
.modal__content {
  position: relative;
  width: min(520px, 100%);
  padding: 28px;
  border-radius: 22px;
  background: rgba(255,255,255,.92);
  color: #0f172a;
  box-shadow: 0 30px 80px rgba(0,0,0,.35);
  transform: translateY(12px) scale(.98);
  transition: transform .25s ease;
}
.modal.is-open .modal__content {
  transform: translateY(0) scale(1);
}
.modal__close {
  position: absolute;
  top: 12px;
  right: 12px;
  border: 0;
  border-radius: 999px;
  padding: 8px 11px;
  cursor: pointer;
}

```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js

```

export class ModalManager {
  constructor() {
    this.openButtons = document.querySelectorAll('[data-open-modal]');
    this.closeButtons = document.querySelectorAll('[data-close-modal]');
    this.activeModal = null;
    this.init();
  }

  init() {
    this.openButtons.forEach((button) => {
      button.addEventListener('click', () => this.open(button.dataset.openModal));
    });

    this.closeButtons.forEach((button) => {
      button.addEventListener('click', () => this.closeActive());
    });

    document.addEventListener('keydown', (event) => {
      if (event.key === 'Escape') this.closeActive();
    });
  }

  open(modalId) {
    const modal = document.getElementById(modalId);
    if (!modal) return;
    modal.classList.add('is-open');
    modal.setAttribute('aria-hidden', 'false');
    document.body.classList.add('no-scroll');
    this.activeModal = modal;
  }

  closeActive() {
    if (!this.activeModal) return;
    this.activeModal.classList.remove('is-open');
    this.activeModal.setAttribute('aria-hidden', 'true');
    document.body.classList.remove('no-scroll');
    this.activeModal = null;
  }
}

// script.js
import { ModalManager } from './funciones.js';
new ModalManager();

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class ModalManager {
  constructor() {
    this.openButtons = document.querySelectorAll('[data-open-modal]');
    this.closeButtons = document.querySelectorAll('[data-close-modal]');
    this.activeModal = null;
    this.init();
  }
  init() {
    this.openButtons.forEach((button) => {
      button.addEventListener('click', () => this.open(button.dataset.openModal));
    });
    this.closeButtons.forEach((button) => {
      button.addEventListener('click', () => this.closeActive());
    });
    document.addEventListener('keydown', (event) => {
      if (event.key === 'Escape') this.closeActive();
    });
  }
  open(modalId) {
    const modal = document.getElementById(modalId);
    if (!modal) return;
    modal.classList.add('is-open');
    modal.setAttribute('aria-hidden', 'false');
    document.body.classList.add('no-scroll');
    this.activeModal = modal;
  }
  closeActive() {
    if (!this.activeModal) return;
    this.activeModal.classList.remove('is-open');
    this.activeModal.setAttribute('aria-hidden', 'true');
    document.body.classList.remove('no-scroll');
    this.activeModal = null;
  }
}
new ModalManager();

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
id="modalContacto"	Identificador unico del modal.	Debe coincidir con data-open-modal="modalContacto".
data-open-modal	Indica que boton abre un modal.	Cambia el valor por el id de tu modal.
data-close-modal	Marca elementos que cierran el modal.	Ponlo en la X y en el overlay si quieres cerrar al hacer clic afuera.
.modal.is-open	Clase de estado abierto.	No la escribes en HTML; JS la agrega y la quita.

Implementacion paso a paso

- 1 Copia el bloque HTML del modal al final del body para que quede por encima del resto del contenido.
- 2 Copia el CSS del modal en styles.css. Si ya tienes un z-index alto en navbar, sube el z-index del modal.
- 3 Copia la clase ModalManager en funciones.js si usas modulos, o en script-normal.js si no usas modulos.
- 4 Crea un boton con data-open-modal y asegúrate de que el valor coincida exactamente con el id del modal.
- 5 Prueba abrir, cerrar con la X, cerrar con el fondo y cerrar con la tecla Escape.

Errores comunes y como solucionarlos

- El modal no abre: revisa que el id del modal y data-open-modal sean iguales.
- El modal queda detras de otros elementos: aumenta z-index en .modal.
- La pagina se sigue moviendo al abrir: agrega .no-scroll { overflow: hidden; } al CSS.

- El boton dentro del modal recarga la pagina: si es un formulario, controla el submit con `preventDefault` cuando sea demo.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con `ToastManager` para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en `localStorage` si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con `aria-label`, `aria-hidden`, `focus-visible` y navegacion por teclado cuando aplique.

27. Toast / notificaciones flotantes

Aspecto	Explicacion
Que hace	Muestra mensajes temporales sin interrumpir al usuario. Es ideal para confirmaciones rapidas.
Donde se usa en industria	Guardar cambios, copiar texto, iniciar sesion, mostrar error de API, confirmar envio de formulario o aviso de carrito.
Como se personaliza	Cambia posicion, duracion, colores por tipo y mensaje.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<button class="btn" data-toast="Datos guardados correctamente" data-toast-type="success">
  Mostrar toast
</button>

<div class="toast-container" id="toastContainer" aria-live="polite"></div>
```

CSS del componente

styles.css

```
.toast-container {
  position: fixed;
  right: 20px;
  bottom: 20px;
  display: grid;
  gap: 12px;
  z-index: 1200;
}
.toast {
  min-width: 260px;
  max-width: 380px;
  padding: 14px 16px;
  border-radius: 16px;
  color: #fff;
  background: rgba(15, 23, 42, .9);
  border: 1px solid rgba(255,255,255,.18);
  box-shadow: 0 16px 45px rgba(0,0,0,.28);
  backdrop-filter: blur(12px);
  animation: toastIn .25s ease both;
}
.toast--success { background: rgba(22, 163, 74, .92); }
.toast--error { background: rgba(220, 38, 38, .92); }
.toast--info { background: rgba(37, 99, 235, .92); }
.toast.is-leaving { animation: toastOut .25s ease both; }
@keyframes toastIn { from { transform: translateY(12px); opacity: 0; } to { transform: translateY(0); opacity: 1; } }
@keyframes toastOut { to { transform: translateY(12px); opacity: 0; } }
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class ToastManager {
  constructor(containerId = 'toastContainer') {
    this.container = document.getElementById(containerId);
  }

  show(message, type = 'info', duration = 3000) {
    if (!this.container) return;

    const toast = document.createElement('div');
    toast.className = `toast toast--${type}`;
    toast.textContent = message;
    this.container.appendChild(toast);

    setTimeout(() => {
      toast.classList.add('is-leaving');
      toast.addEventListener('animationend', () => toast.remove(), { once: true });
    }, duration);
  }

  bindButtons() {
    document.querySelectorAll('[data-toast]').forEach((button) => {
      button.addEventListener('click', () => {
        this.show(button.dataset.toast, button.dataset.toastType || 'info');
      });
    });
  }
}

// script.js
import { ToastManager } from './funciones.js';
const toast = new ToastManager();
toast.bindButtons();
// toast.show('Producto agregado', 'success');
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class ToastManager {
  constructor(containerId = 'toastContainer') {
    this.container = document.getElementById(containerId);
  }
  show(message, type = 'info', duration = 3000) {
    if (!this.container) return;
    const toast = document.createElement('div');
    toast.className = `toast toast--${type}`;
    toast.textContent = message;
    this.container.appendChild(toast);
    setTimeout(() => {
      toast.classList.add('is-leaving');
      toast.addEventListener('animationend', () => toast.remove(), { once: true });
    }, duration);
  }
  bindButtons() {
    document.querySelectorAll('[data-toast]').forEach((button) => {
      button.addEventListener('click', () => {
        this.show(button.dataset.toast, button.dataset.toastType || 'info');
      });
    });
  }
}

const toast = new ToastManager();
toast.bindButtons();
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
toastContainer	Contenedor donde se insertan los mensajes.	Cambia el id si quieres varios contenedores.
data-toast	Texto del mensaje.	Personaliza el texto visible.
data-toast-type	Tipo visual: success, error, info.	Crea mas clases CSS si necesitas warning u otros.
duration	Tiempo antes de desaparecer.	Aumenta a 5000 si el mensaje es largo.

Implementacion paso a paso

1. Agrega el contenedor toastContainer al final del body.
2. Copia los estilos del toast y ajusta right/bottom si lo quieres arriba o a la izquierda.
3. Importa ToastManager o pega la version normal.
4. Usa botones con data-toast o llama toast.show() desde otros eventos.
5. Prueba varios toasts seguidos para confirmar que se apilan bien.

Errores comunes y como solucionarlos

- No aparece nada: falta el div con id toastContainer.
- Sale sin color: revisa que el tipo coincida con una clase .toast--tipo.
- No se elimina: revisa que el navegador ejecute la animacion o reemplaza animationend por toast.remove en setTimeout.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

28. Carrusel / slider

Aspecto	Explicacion
Que hace	Permite mostrar varios bloques de contenido, imagenes o promociones en un mismo espacio con botones anterior/siguiente.
Donde se usa en industria	Hero de home, testimonios, productos destacados, galeria de imagenes, banners comerciales.
Como se personaliza	Duplica slides, cambia imagenes, textos, velocidad, autoplay y controles.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<section class="carousel" id="mainCarousel">
  <div class="carousel__track">
    <article class="carousel__slide is-active">Slide 1 - Promocion principal</article>
    <article class="carousel__slide">Slide 2 - Producto destacado</article>
    <article class="carousel__slide">Slide 3 - Testimonio</article>
  </div>

  <button class="carousel__btn carousel__btn--prev" type="button" data-carousel-prev>Anterior</button>
  <button class="carousel__btn carousel__btn--next" type="button" data-carousel-next>Siguiente</button>

  <div class="carousel__dots" data-carousel-dots></div>
</section>
```

CSS del componente

styles.css

```

.carousel {
  position: relative;
  overflow: hidden;
  border-radius: 24px;
  background: rgba(255,255,255,.12);
  border: 1px solid rgba(255,255,255,.18);
}
.carousel__track { position: relative; min-height: 240px; }
.carousel__slide {
  position: absolute;
  inset: 0;
  display: grid;
  place-items: center;
  padding: 32px;
  opacity: 0;
  transform: translateX(20px);
  transition: opacity .35s ease, transform .35s ease;
}
.carousel__slide.is-active {
  opacity: 1;
  transform: translateX(0);
  position: relative;
}
.carousel__btn {
  position: absolute;
  top: 50%;
  transform: translateY(-50%);
  border: 0;
  border-radius: 999px;
  padding: 10px 14px;
  cursor: pointer;
}
.carousel__btn--prev { left: 12px; }
.carousel__btn--next { right: 12px; }
.carousel__dots {
  display: flex;
  justify-content: center;
  gap: 8px;
  padding: 12px;
}
.carousel__dot {
  width: 10px;
  height: 10px;
  border-radius: 999px;
  border: 0;
  background: rgba(255,255,255,.35);
  cursor: pointer;
}
.carousel__dot.is-active { background: #2563eb; }

```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js

```

export class Carousel {
  constructor(carouselId, { autoplay = false, interval = 4000 } = {}) {
    this.carousel = document.getElementById(carouselId);
    if (!this.carousel) return;
    this.slides = [...this.carousel.querySelectorAll('.carousel__slide')];
    this.prev = this.carousel.querySelector('[data-carousel-prev]');
    this.next = this.carousel.querySelector('[data-carousel-next]');
    this.dotsContainer = this.carousel.querySelector('[data-carousel-dots]');
    this.index = 0;
    this.autoplay = autoplay;
    this.interval = interval;
    this.timer = null;
    this.init();
  }

  init() {
    this.renderDots();
    this.prev?.addEventListener('click', () => this.goTo(this.index - 1));
    this.next?.addEventListener('click', () => this.goTo(this.index + 1));
    if (this.autoplay) this.start();
  }

  renderDots() {
    this.dotsContainer.innerHTML = this.slides.map((_, i) =>
      `<button class="carousel__dot" data-dot="${i}" aria-label="Ir al slide ${i + 1}"></button>`
    ).join('');
    this.dots = [...this.dotsContainer.querySelectorAll('.carousel__dot')];
    this.dots.forEach((dot) => dot.addEventListener('click', () => this.goTo(Number(dot.dataset.
dot))));
    this.update();
  }

  goTo(newIndex) {
    this.index = (newIndex + this.slides.length) % this.slides.length;
    this.update();
  }

  update() {
    this.slides.forEach((slide, i) => slide.classList.toggle('is-active', i === this.index));
    this.dots?.forEach((dot, i) => dot.classList.toggle('is-active', i === this.index));
  }

  start() { this.timer = setInterval(() => this.goTo(this.index + 1), this.interval); }
  stop() { clearInterval(this.timer); }
}

// script.js
import { Carousel } from './funciones.js';
new Carousel('mainCarousel', { autoplay: true, interval: 5000 });

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class Carousel {
  constructor(carouselId, options = {}) {
    this.carousel = document.getElementById(carouselId);
    if (!this.carousel) return;
    this.slides = [...this.carousel.querySelectorAll('.carousel__slide')];
    this.prev = this.carousel.querySelector('[data-carousel-prev]');
    this.next = this.carousel.querySelector('[data-carousel-next]');
    this.dotsContainer = this.carousel.querySelector('[data-carousel-dots]');
    this.index = 0;
    this.autoplay = options.autoplay || false;
    this.interval = options.interval || 4000;
    this.init();
  }
  init() {
    this.renderDots();
    this.prev?.addEventListener('click', () => this.goTo(this.index - 1));
    this.next?.addEventListener('click', () => this.goTo(this.index + 1));
    if (this.autoplay) setInterval(() => this.goTo(this.index + 1), this.interval);
  }
  renderDots() {
    this.dotsContainer.innerHTML = this.slides.map((_, i) => `<button class="carousel__dot" data-
dot="${i}"></button>`).join('');
    this.dots = [...this.dotsContainer.querySelectorAll('.carousel__dot')];
    this.dots.forEach((dot) => dot.addEventListener('click', () => this.goTo(Number(dot.dataset.
dot))));
    this.update();
  }
  goTo(newIndex) {
    this.index = (newIndex + this.slides.length) % this.slides.length;
    this.update();
  }
  update() {
    this.slides.forEach((slide, i) => slide.classList.toggle('is-active', i === this.index));
    this.dots.forEach((dot, i) => dot.classList.toggle('is-active', i === this.index));
  }
}
new Carousel('mainCarousel', { autoplay: true });

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
mainCarousel	Id del carrusel.	Cambia el id si tienes mas de un carrusel.
.carousel__slide	Cada elemento visible del carrusel.	Duplica esta etiqueta para agregar mas slides.
autoplay	Activa cambio automatico.	Pon false para que solo cambie con botones.
interval	Tiempo entre slides.	Valor en milisegundos: 5000 = 5 segundos.

Implementacion paso a paso

- 1 Copia el HTML del carrusel donde quieras mostrarlo.
- 2 Duplica o elimina elementos .carousel__slide segun el numero de contenidos.
- 3 Copia los estilos y ajusta min-height si tus imagenes son mas grandes.
- 4 Inicializa la clase con el id correspondiente.
- 5 Prueba botones, dots y autoplay.

Errores comunes y como solucionarlos

- Solo se ve un slide siempre: revisa que la clase is-active cambie en DevTools.
- Los botones no aparecen: puede estar faltando position: relative en .carousel.
- Los dots no se generan: falta el contenedor data-carousel-dots.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

29. Tooltip / ayuda contextual

Aspecto	Explicacion
Que hace	Muestra una ayuda corta al pasar el mouse o enfocar un elemento.
Donde se usa en industria	Iconos de ayuda, botones con acciones, formularios, tablas con estados, paneles administrativos.
Como se personaliza	Cambia el texto de data-tooltip, la posicion y el estilo visual.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<button class="icon-button" data-tooltip="Editar producto" aria-label="Editar producto">
  Editar
</button>

<span class="info" data-tooltip="Este dato se guarda automaticamente">?</span>
```

CSS del componente

styles.css

```

[data-tooltip] {
  position: relative;
}
[data-tooltip]::before,
[data-tooltip]::after {
  position: absolute;
  left: 50%;
  opacity: 0;
  visibility: hidden;
  pointer-events: none;
  transition: opacity .2s ease, transform .2s ease;
  z-index: 900;
}
[data-tooltip]::before {
  content: attr(data-tooltip);
  bottom: calc(100% + 10px);
  transform: translateX(-50%) translateY(6px);
  width: max-content;
  max-width: 220px;
  padding: 8px 10px;
  border-radius: 10px;
  background: #0f172a;
  color: #fff;
  font-size: 13px;
  box-shadow: 0 10px 25px rgba(0,0,0,.22);
}
[data-tooltip]::after {
  content: "";
  bottom: calc(100% + 4px);
  transform: translateX(-50%) translateY(6px);
  border: 6px solid transparent;
  border-top-color: #0f172a;
}
[data-tooltip]:hover::before,
[data-tooltip]:hover::after,
[data-tooltip]:focus-visible::before,
[data-tooltip]:focus-visible::after {
  opacity: 1;
  visibility: visible;
  transform: translateX(-50%) translateY(0);
}

```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

// Version modular opcional para actualizar tooltips dinamicamente.
export function setTooltip(selector, text) {
  const element = document.querySelector(selector);
  if (!element) return;
  element.setAttribute('data-tooltip', text);
}

export function removeTooltip(selector) {
  const element = document.querySelector(selector);
  if (!element) return;
  element.removeAttribute('data-tooltip');
}

// script.js
import { setTooltip } from './funciones.js';
setTooltip('#botonGuardar', 'Guarda los cambios del formulario');

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
function setTooltip(selector, text) {
  const element = document.querySelector(selector);
  if (!element) return;
  element.setAttribute('data-tooltip', text);
}

function removeTooltip(selector) {
  const element = document.querySelector(selector);
  if (!element) return;
  element.removeAttribute('data-tooltip');
}

setTooltip('#botonGuardar', 'Guarda los cambios del formulario');
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
data-tooltip	Texto del tooltip.	Cambia el texto directamente en HTML.
bottom	Ubicacion vertical del tooltip.	Cambia a top/left/right si quieres otra posicion.
max-width	Ancho maximo del tooltip.	Sube a 280px si el texto es largo.
z-index	Prioridad visual.	Aumenta si el tooltip queda detras de una card.

Implementacion paso a paso

1. Agrega data-tooltip al elemento que necesita ayuda contextual.
2. Asegurate de que el elemento pueda recibir foco si es interactivo.
3. Copia el CSS y prueba hover y focus-visible.
4. Si el texto es largo, acortalo o aumenta max-width.
5. Usa setTooltip solo si necesitas cambiar el texto desde JavaScript.

Errores comunes y como solucionarlos

- No se ve el tooltip: puede estar recortado por un contenedor con overflow: hidden.
- Se ve muy ancho: usa max-width o divide el texto.
- No funciona en celular: para mobile conviene mostrar ayuda con clic o modal.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.

- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

30. Badge / indicador numerico

Aspecto	Explicacion
Que hace	Muestra un numero o estado pequeño sobre un boton, icono o texto.
Donde se usa en industria	Notificaciones, carrito de compras, mensajes pendientes, tareas abiertas, alertas de panel administrativo.
Como se personaliza	Cambia el id del badge, el numero, los colores y la condicion para ocultarlo.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<button class="notification-button" type="button">
  Notificaciones
  <span class="badge" id="notificationBadge">3</span>
</button>

<button class="btn" id="addNotification">Agregar</button>
<button class="btn" id="clearNotifications">Limpiar</button>
```

CSS del componente

styles.css

```
.notification-button {
  position: relative;
  display: inline-flex;
  align-items: center;
  gap: 8px;
  padding: 10px 16px;
  border-radius: 999px;
}
.badge {
  min-width: 22px;
  height: 22px;
  display: inline-grid;
  place-items: center;
  padding: 0 6px;
  border-radius: 999px;
  background: #ef4444;
  color: #fff;
  font-size: 12px;
  font-weight: 700;
  box-shadow: 0 6px 16px rgba(239,68,68,.35);
}
.badge.is-hidden {
  display: none;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class BadgeCounter {
  constructor(badgeId, initialValue = 0) {
    this.badge = document.getElementById(badgeId);
    this.value = initialValue;
    this.render();
  }

  increment(amount = 1) {
    this.value += amount;
    this.render();
  }

  decrement(amount = 1) {
    this.value = Math.max(0, this.value - amount);
    this.render();
  }

  reset() {
    this.value = 0;
    this.render();
  }

  render() {
    if (!this.badge) return;
    this.badge.textContent = this.value > 99 ? '99+' : this.value;
    this.badge.classList.toggle('is-hidden', this.value === 0);
  }
}

// script.js
import { BadgeCounter } from './funciones.js';
const badge = new BadgeCounter('notificationBadge', 3);
document.getElementById('addNotification').addEventListener('click', () => badge.increment());
document.getElementById('clearNotifications').addEventListener('click', () => badge.reset());
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class BadgeCounter {
  constructor(badgeId, initialValue = 0) {
    this.badge = document.getElementById(badgeId);
    this.value = initialValue;
    this.render();
  }
  increment(amount = 1) { this.value += amount; this.render(); }
  decrement(amount = 1) { this.value = Math.max(0, this.value - amount); this.render(); }
  reset() { this.value = 0; this.render(); }
  render() {
    if (!this.badge) return;
    this.badge.textContent = this.value > 99 ? '99+' : this.value;
    this.badge.classList.toggle('is-hidden', this.value === 0);
  }
}

const badge = new BadgeCounter('notificationBadge', 3);
document.getElementById('addNotification').addEventListener('click', () => badge.increment());
document.getElementById('clearNotifications').addEventListener('click', () => badge.reset());
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
notificationBadge	Id del span que muestra el numero.	Cambia si tienes varios badges.
initialValue	Valor inicial.	Usa 0 si debe iniciar oculto.
99+	Limite visual.	Cambia a 9+ o 999+ segun tu interfaz.

Elemento	Para que sirve	Que debes cambiar
.badge.is-hidden	Clase que oculta el indicador.	Cambia display none por opacity si quieres animacion.

Implementacion paso a paso

- 1 Crea un span con clase badge dentro del boton o icono.
- 2 Copia los estilos del badge.
- 3 Inicializa BadgeCounter con el id correcto.
- 4 Llama increment, decrement o reset segun la accion del usuario.
- 5 Prueba que al llegar a 0 se oculte.

Errores comunes y como solucionarlos

- El badge aparece fuera de lugar: pon position: relative en el contenedor.
- No se actualiza: revisa que el id sea correcto.
- Muestra valores negativos: usa Math.max como en el ejemplo.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

31. Copy clipboard / copiar al portapapeles

Aspecto	Explicacion
Que hace	Permite copiar texto con un clic desde un bloque, input, codigo o cupon.
Donde se usa en industria	Cupones, enlaces de invitacion, codigos, tokens publicos, datos de contacto, comandos de instalacion.
Como se personaliza	Cambia el selector data-copy, el mensaje de confirmacion y el fallback.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<p id="codigoCupon">SIERRA-2026</p>
<button class="btn" data-copy="#codigoCupon">Copiar cupon</button>

<input id="correoEmpresa" value="contacto@empresa.com" readonly>
<button class="btn" data-copy="#correoEmpresa">Copiar correo</button>
```

CSS del componente

styles.css

```
[data-copy] {
  cursor: pointer;
}
.copy-feedback {
  display: inline-block;
  margin-left: 8px;
  color: #16a34a;
  font-weight: 700;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js


```

export class ClipboardManager {
  constructor({ onSuccess = null, onError = null } = {}) {
    this.onSuccess = onSuccess;
    this.onError = onError;
    this.init();
  }

  init() {
    document.querySelectorAll('[data-copy]').forEach((button) => {
      button.addEventListener('click', () => this.copyFromSelector(button.dataset.copy, button));
    });
  }

  async copyFromSelector(selector, trigger) {
    const target = document.querySelector(selector);
    if (!target) return;
    const text = target.value ?? target.textContent;

    try {
      await navigator.clipboard.writeText(text.trim());
      trigger.textContent = 'Copiado';
      setTimeout(() => trigger.textContent = 'Copiar', 1500);
      this.onSuccess?.(text);
    } catch (error) {
      this.fallbackCopy(text);
      this.onError?.(error);
    }
  }

  fallbackCopy(text) {
    const area = document.createElement('textarea');
    area.value = text;
    document.body.appendChild(area);
    area.select();
    document.execCommand('copy');
    area.remove();
  }
}

// script.js
import { ClipboardManager } from './funciones.js';
new ClipboardManager();

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class ClipboardManager {
  constructor() { this.init(); }
  init() {
    document.querySelectorAll('[data-copy]').forEach((button) => {
      button.addEventListener('click', () => this.copyFromSelector(button.dataset.copy, button));
    });
  }
  async copyFromSelector(selector, trigger) {
    const target = document.querySelector(selector);
    if (!target) return;
    const text = target.value || target.textContent;
    try {
      await navigator.clipboard.writeText(text.trim());
      trigger.textContent = 'Copiado';
      setTimeout(() => trigger.textContent = 'Copiar', 1500);
    } catch (error) {
      const area = document.createElement('textarea');
      area.value = text;
      document.body.appendChild(area);
      area.select();
      document.execCommand('copy');
      area.remove();
    }
  }
}

new ClipboardManager();

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
data-copy	Selector del elemento que contiene el texto.	Debe apuntar a un id, clase o selector valido.
navigator.clipboard	API moderna de copia.	Funciona mejor en HTTPS o localhost.
trigger.textContent	Texto del boton despues de copiar.	Personaliza Copiado/Copiar segun tu UI.
fallbackCopy	Plan B para navegadores antiguos.	Mantenlo si quieres compatibilidad.

Implementacion paso a paso

- 1 Pon el texto a copiar en un elemento con id.
- 2 Crea un boton con data-copy apuntando a ese selector.
- 3 Copia la clase ClipboardManager y ejecútala.
- 4 Prueba en localhost o servidor seguro.
- 5 Conecta con ToastManager si quieres mostrar una notificacion bonita.

Errores comunes y como solucionarlos

- No copia: el selector data-copy no encuentra el elemento.
- Funciona local pero no en servidor: revisa permisos HTTPS.
- Cambia el texto del boton a Copiado pero no vuelve: revisa setTimeout.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

32. Cards dinamicas desde un array

Aspecto	Explicacion
Que hace	Renderiza tarjetas automaticamente a partir de datos JavaScript.
Donde se usa en industria	Catalogos, portafolios, productos, cursos, blogs, equipos, servicios y dashboards.
Como se personaliza	Edita el array de datos, el template HTML y el contenedor donde se pintan las cards.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<section class="cards-grid" id="cardsGrid"></section>
```

CSS del componente

styles.css

```
.cards-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));  
  gap: 18px;  
}  
.card {  
  padding: 20px;  
  border-radius: 20px;  
  background: rgba(255,255,255,.12);  
  border: 1px solid rgba(255,255,255,.18);  
  box-shadow: 0 14px 35px rgba(0,0,0,.18);  
}  
.card__title { margin: 0 0 8px; font-size: 20px; }  
.card__meta { color: #64748b; font-size: 14px; }  
.card__price { font-weight: 800; color: #2563eb; }
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export class CatalogCards {
  constructor(containerId, items = []) {
    this.container = document.getElementById(containerId);
    this.items = items;
    this.render(this.items);
  }

  setItems(items) {
    this.items = items;
    this.render(this.items);
  }

  render(items) {
    if (!this.container) return;
    if (items.length === 0) {
      this.container.innerHTML = '<p>No hay resultados disponibles.</p>';
      return;
    }

    this.container.innerHTML = items.map((item) => `
      <article class="card" data-category="${item.category}">
        <h3 class="card__title">${item.name}</h3>
        <p>${item.description}</p>
        <p class="card__meta">Categoria: ${item.category}</p>
        <p class="card__price">${item.price}</p>
      </article>
    `).join('');
  }
}

// script.js
import { CatalogCards } from './funciones.js';
const products = [
  { name: 'Producto A', category: 'web', price: 12000, description: 'Descripcion corta.' },
  { name: 'Producto B', category: 'data', price: 18000, description: 'Otra descripcion.' }
];
const catalog = new CatalogCards('cardsGrid', products);

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class CatalogCards {
  constructor(containerId, items = []) {
    this.container = document.getElementById(containerId);
    this.items = items;
    this.render(this.items);
  }
  setItems(items) {
    this.items = items;
    this.render(this.items);
  }
  render(items) {
    if (!this.container) return;
    if (items.length === 0) {
      this.container.innerHTML = '<p>No hay resultados disponibles.</p>';
      return;
    }
    this.container.innerHTML = items.map((item) => `
      <article class="card" data-category="${item.category}">
        <h3 class="card__title">${item.name}</h3>
        <p>${item.description}</p>
        <p class="card__meta">Categoria: ${item.category}</p>
        <p class="card__price">${item.price}</p>
      </article>
    `).join('');
  }
}
const products = [
  { name: 'Producto A', category: 'web', price: 12000, description: 'Descripcion corta.' },
  { name: 'Producto B', category: 'data', price: 18000, description: 'Otra descripcion.' }
];
const catalog = new CatalogCards('cardsGrid', products);

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
cardsGrid	Contenedor donde se pintan las cards.	Cambia el id si tu seccion tiene otro nombre.
products/items	Array de datos.	Agrega campos como image, rating, stock o url.
render()	Metodo que convierte datos en HTML.	Edita el template para cambiar el diseño.
data-category	Dato util para filtrar.	Debe coincidir con tus filtros chips.

Implementacion paso a paso

- 1 Crea un contenedor vacio para las cards.
- 2 Define un array con objetos consistentes.
- 3 Copia la clase CatalogCards.
- 4 Instancia la clase con el id y el array.
- 5 Edita el template dentro de render para adaptarlo a tu proyecto.

Errores comunes y como solucionarlos

- Se imprime undefined: falta una propiedad en algun objeto del array.
- No aparecen cards: revisa el id del contenedor.
- Hay riesgo de HTML no confiable si los datos vienen de usuarios. Sanitiza si viene de fuentes externas.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

33. Búsqueda con debounce

Aspecto	Explicacion
Que hace	Evita ejecutar una búsqueda en cada tecla inmediatamente. Espera unos milisegundos despues de que el usuario deja de escribir.
Donde se usa en industria	Filtros de catalogo, buscadores, tablas, llamadas a APIs, dashboards y autocompletados.
Como se personaliza	Cambia el delay, el input, la lista de datos y la funcion que filtra.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<input type="search" id="searchInput" placeholder="Buscar productos...">
<section id="cardsGrid" class="cards-grid"></section>
```

CSS del componente

styles.css

```
input[type="search"] {
  width: 100%;
  padding: 12px 14px;
  border-radius: 14px;
  border: 1px solid #cbd5e1;
  margin-bottom: 18px;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js

```

export function debounce(callback, delay = 300) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => callback(...args), delay);
  };
}

export class SearchController {
  constructor(inputId, items, onResults, delay = 300) {
    this.input = document.getElementById(inputId);
    this.items = items;
    this.onResults = onResults;
    this.delay = delay;
    this.init();
  }

  init() {
    if (!this.input) return;
    this.input.addEventListener('input', debounce(() => this.search(), this.delay));
  }

  search() {
    const term = this.input.value.trim().toLowerCase();
    const results = this.items.filter((item) => {
      return item.name.toLowerCase().includes(term)
        || item.category.toLowerCase().includes(term)
        || item.description.toLowerCase().includes(term);
    });
    this.onResults(results);
  }
}

// script.js
import { SearchController, CatalogCards } from './funciones.js';
const catalog = new CatalogCards('cardsGrid', products);
new SearchController('searchInput', products, (results) => catalog.render(results), 350);

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

function debounce(callback, delay = 300) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => callback(...args), delay);
  };
}

class SearchController {
  constructor(inputId, items, onResults, delay = 300) {
    this.input = document.getElementById(inputId);
    this.items = items;
    this.onResults = onResults;
    this.delay = delay;
    this.init();
  }

  init() {
    if (!this.input) return;
    this.input.addEventListener('input', debounce(() => this.search(), this.delay));
  }

  search() {
    const term = this.input.value.trim().toLowerCase();
    const results = this.items.filter((item) =>
      item.name.toLowerCase().includes(term) ||
      item.category.toLowerCase().includes(term) ||
      item.description.toLowerCase().includes(term)
    );
    this.onResults(results);
  }
}

new SearchController('searchInput', products, (results) => catalog.render(results), 350);

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
delay	Tiempo de espera para buscar.	300 a 500 ms suele sentirse bien.
items	Datos originales.	Debe ser el array completo, no el ya filtrado.
onResults	Funcion que recibe resultados.	Puedes renderizar cards, tabla o lista.
term	Texto escrito por el usuario.	Puedes normalizar acentos si lo necesitas.

Implementacion paso a paso

- 1 Crea un input tipo search.
- 2 Ten un array original sin modificar.
- 3 Copia debounce y SearchController.
- 4 Pasa una funcion callback que actualice la UI.
- 5 Prueba con mayusculas, minusculas y texto vacio.

Errores comunes y como solucionarlos

- Filtra sobre resultados ya filtrados y pierde datos: siempre filtra desde el array original.
- Busca muy lento: baja el delay.
- Busca demasiado: sube el delay si hace llamadas a una API.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

34. Filtros chips por categoria

Aspecto	Explicacion
Que hace	Permite filtrar datos con botones pequeños tipo etiqueta o categoria.
Donde se usa en industria	Catalogos, portafolios, blogs, dashboards, filtros de productos, categorias de servicios.
Como se personaliza	Cambia data-category y asegúrate de que coincida con la categoria del dato.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<div class="chips" id="categoryChips">
  <button class="chip is-active" data-category="all">Todos</button>
  <button class="chip" data-category="web">Web</button>
  <button class="chip" data-category="data">Datos</button>
  <button class="chip" data-category="design">Diseno</button>
</div>

<section id="cardsGrid" class="cards-grid"></section>
```

CSS del componente

styles.css

```
.chips {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  margin-bottom: 18px;
}
.chip {
  border: 1px solid #cbd5e1;
  border-radius: 999px;
  padding: 9px 14px;
  background: #fff;
  cursor: pointer;
  transition: background .2s ease, transform .2s ease;
}
.chip:hover { transform: translateY(-1px); }
.chip.is-active {
  background: linear-gradient(135deg, #2563eb, #7c3aed);
  color: #fff;
  border-color: transparent;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export class ChipFilters {
  constructor(containerId, items, onResults) {
    this.container = document.getElementById(containerId);
    this.items = items;
    this.onResults = onResults;
    this.activeCategory = 'all';
    this.init();
  }

  init() {
    if (!this.container) return;
    this.container.addEventListener('click', (event) => {
      const chip = event.target.closest('[data-category]');
      if (!chip) return;
      this.activeCategory = chip.dataset.category;
      this.container.querySelectorAll('.chip').forEach((item) => item.classList.remove('is-
active'));
      chip.classList.add('is-active');
      this.apply();
    });
  }

  apply() {
    const results = this.activeCategory === 'all'
      ? this.items
      : this.items.filter((item) => item.category === this.activeCategory);
    this.onResults(results);
  }
}

// script.js
import { ChipFilters } from './funciones.js';
new ChipFilters('categoryChips', products, (results) => catalog.render(results));

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class ChipFilters {
  constructor(containerId, items, onResults) {
    this.container = document.getElementById(containerId);
    this.items = items;
    this.onResults = onResults;
    this.activeCategory = 'all';
    this.init();
  }

  init() {
    if (!this.container) return;
    this.container.addEventListener('click', (event) => {
      const chip = event.target.closest('[data-category]');
      if (!chip) return;
      this.activeCategory = chip.dataset.category;
      this.container.querySelectorAll('.chip').forEach((item) => item.classList.remove('is-
active'));
      chip.classList.add('is-active');
      this.apply();
    });
  }

  apply() {
    const results = this.activeCategory === 'all' ? this.items : this.items.filter((item) => item.
category === this.activeCategory);
    this.onResults(results);
  }
}

new ChipFilters('categoryChips', products, (results) => catalog.render(results));

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
data-category	Categoría que representa cada chip.	Debe coincidir con item.category.
all	Categoría especial para mostrar todos.	Puedes cambiarla por todos, pero actualiza JS.

Elemento	Para que sirve	Que debes cambiar
is-active	Clase visual del filtro activo.	Cambia colores en CSS.
onResults	Funcion que actualiza la vista.	Renderiza cards, tabla o lista.

Implementacion paso a paso

- 1 Crea un contenedor de chips.
- 2 Agrega un boton por categoria.
- 3 Asegúrate de que las categorias existan en los datos.
- 4 Inicializa ChipFilters con el contenedor, datos y callback.
- 5 Prueba cada chip y revisa que el activo cambie visualmente.

Errores comunes y como solucionarlos

- Un chip no filtra: data-category no coincide exactamente con item.category.
- No se ve el activo: falta la clase CSS .chip.is-active.
- Al mezclar con busqueda, necesitas un estado global que combine filtros y texto.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

35. Ordenamiento de resultados

Aspecto	Explicacion
Que hace	Ordena listas por precio, nombre, calificacion, fecha u otro criterio.
Donde se usa en industria	E-commerce, catalogos, dashboards, tablas, listados de usuarios y reportes.
Como se personaliza	Cambia las opciones del select y la funcion de comparacion.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<label for="sortSelect">Ordenar por:</label>
<select id="sortSelect">
  <option value="name-asc">Nombre A-Z</option>
  <option value="name-desc">Nombre Z-A</option>
  <option value="price-asc">Menor precio</option>
  <option value="price-desc">Mayor precio</option>
</select>
```

CSS del componente

styles.css

```
#sortSelect {
  width: 100%;
  max-width: 280px;
  padding: 10px 12px;
  border-radius: 12px;
  border: 1px solid #cbd5e1;
  background: #fff;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export class SortController {
  constructor(selectId, items, onResults) {
    this.select = document.getElementById(selectId);
    this.items = items;
    this.onResults = onResults;
    this.init();
  }

  init() {
    if (!this.select) return;
    this.select.addEventListener('change', () => this.sort());
  }

  sort() {
    const value = this.select.value;
    const sorted = [...this.items];

    const strategies = {
      'name-asc': (a, b) => a.name.localeCompare(b.name),
      'name-desc': (a, b) => b.name.localeCompare(a.name),
      'price-asc': (a, b) => a.price - b.price,
      'price-desc': (a, b) => b.price - a.price,
    };

    sorted.sort(strategies[value] || strategies['name-asc']);
    this.onResults(sorted);
  }
}

// script.js
import { SortController } from './funciones.js';
new SortController('sortSelect', products, (results) => catalog.render(results));

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class SortController {
  constructor(selectId, items, onResults) {
    this.select = document.getElementById(selectId);
    this.items = items;
    this.onResults = onResults;
    this.init();
  }

  init() {
    if (!this.select) return;
    this.select.addEventListener('change', () => this.sort());
  }

  sort() {
    const value = this.select.value;
    const sorted = [...this.items];
    const strategies = {
      'name-asc': (a, b) => a.name.localeCompare(b.name),
      'name-desc': (a, b) => b.name.localeCompare(a.name),
      'price-asc': (a, b) => a.price - b.price,
      'price-desc': (a, b) => b.price - a.price,
    };
    sorted.sort(strategies[value] || strategies['name-asc']);
    this.onResults(sorted);
  }
}

new SortController('sortSelect', products, (results) => catalog.render(results));

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
sortSelect	Id del select de ordenamiento.	Cambia si tienes otro id.
strategies	Objeto con funciones de orden.	Agrega mas criterios: rating, date, stock.
[...this.items]	Copia del array original.	Evita modificar directamente los datos base.
localeCompare	Orden alfabético.	Muy util para nombres y textos.

Implementacion paso a paso

- 1 Crea un select con opciones de orden.
- 2 Cada value debe existir como clave dentro de strategies.
- 3 Copia SortController.
- 4 Pasa el array y un callback de render.
- 5 Prueba orden ascendente y descendente.

Errores comunes y como solucionarlos

- El orden no cambia: el value del option no coincide con strategies.
- Se dañan los datos originales: no uses sort directamente sobre items si luego necesitas el orden original.
- Ordena numeros como texto: usa resta $a.price - b.price$ para valores numericos.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

36. Paginacion

Aspecto	Explicacion
Que hace	Divide muchos resultados en varias paginas para evitar interfaces pesadas.
Donde se usa en industria	Catalogos, tablas administrativas, blogs, busquedas, resultados de API y listados grandes.
Como se personaliza	Cambia perPage, contenedor de botones y render de resultados.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<section id="cardsGrid" class="cards-grid"></section>
<nav class="pagination" id="pagination" aria-label="Paginacion"></nav>
```

CSS del componente

styles.css

```
.pagination {
  display: flex;
  justify-content: center;
  flex-wrap: wrap;
  gap: 8px;
  margin-top: 20px;
}
.pagination__button {
  border: 1px solid #cbd5e1;
  background: #fff;
  border-radius: 10px;
  padding: 8px 12px;
  cursor: pointer;
}
.pagination__button.is-active {
  color: #fff;
  background: #2563eb;
  border-color: #2563eb;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export class Pagination {
  constructor(containerId, items, perPage, onPageChange) {
    this.container = document.getElementById(containerId);
    this.items = items;
    this.perPage = perPage;
    this.onPageChange = onPageChange;
    this.currentPage = 1;
    this.render();
    this.emit();
  }

  setItems(items) {
    this.items = items;
    this.currentPage = 1;
    this.render();
    this.emit();
  }

  get totalPages() {
    return Math.ceil(this.items.length / this.perPage) || 1;
  }

  getPageItems() {
    const start = (this.currentPage - 1) * this.perPage;
    return this.items.slice(start, start + this.perPage);
  }

  render() {
    if (!this.container) return;
    this.container.innerHTML = Array.from({ length: this.totalPages }, (_, i) => `
      <button class="pagination__button ${i + 1 === this.currentPage ? 'is-active' : ''}" data-
page="${i + 1}">
        ${i + 1}
      </button>
    `).join('');
    this.container.querySelectorAll('[data-page]').forEach((button) => {
      button.addEventListener('click', () => {
        this.currentPage = Number(button.dataset.page);
        this.render();
        this.emit();
      });
    });
  }

  emit() {
    this.onPageChange(this.getPageItems(), this.currentPage, this.totalPages);
  }
}

// script.js
import { Pagination } from './funciones.js';
const pager = new Pagination('pagination', products, 6, (pageItems) => catalog.render(pageItems));

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js


```

class Pagination {
  constructor(containerId, items, perPage, onPageChange) {
    this.container = document.getElementById(containerId);
    this.items = items;
    this.perPage = perPage;
    this.onPageChange = onPageChange;
    this.currentPage = 1;
    this.render();
    this.emit();
  }
  setItems(items) { this.items = items; this.currentPage = 1; this.render(); this.emit(); }
  get totalPages() { return Math.ceil(this.items.length / this.perPage) || 1; }
  getPageItems() {
    const start = (this.currentPage - 1) * this.perPage;
    return this.items.slice(start, start + this.perPage);
  }
  render() {
    if (!this.container) return;
    this.container.innerHTML = Array.from({ length: this.totalPages }, (_, i) => `<button
class="pagination__button ${i + 1} === this.currentPage ? 'is-active' : ''" data-page="${i + 1}">${i +
1}</button>`).join('');
    this.container.querySelectorAll('[data-page]').forEach((button) => {
      button.addEventListener('click', () => {
        this.currentPage = Number(button.dataset.page);
        this.render();
        this.emit();
      });
    });
  }
  emit() { this.onPageChange(this.getPageItems(), this.currentPage, this.totalPages); }
}
const pager = new Pagination('pagination', products, 6, (pageItems) => catalog.render(pageItems));

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
pagination	Contenedor de botones de pagina.	Cambia id segun tu HTML.
perPage	Cantidad de elementos por pagina.	6, 9 o 12 son comunes para cards.
setItems()	Actualiza resultados paginados.	Usalo despues de buscar o filtrar.
slice	Extrae solo los elementos visibles.	No modifica el array original.

Implementacion paso a paso

- 1 Crea el contenedor de resultados y el nav de paginacion.
- 2 Copia la clase Pagination.
- 3 Pasa el array completo, cantidad por pagina y callback.
- 4 Cuando filtres o busques, llama pager.setItems(resultados).
- 5 Prueba ultima pagina y filtros con pocos resultados.

Errores comunes y como solucionarlos

- Se muestran resultados de otra pagina tras filtrar: debes reiniciar currentPage a 1.
- No aparecen botones: totalPages es 0 si el array no llega correctamente.
- Demasiados botones: para listas enormes conviene paginacion con puntos suspensivos.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.

- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

37. Tabla ordenable por columnas

Aspecto	Explicacion
Que hace	Permite ordenar filas al hacer clic en los encabezados de una tabla.
Donde se usa en industria	Paneles administrativos, reportes, inventarios, usuarios, facturas, indicadores y listados tecnicos.
Como se personaliza	Agrega data-sort-table a los th y define si la columna es texto o numero.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<table class="data-table" id="ordersTable">
  <thead>
    <tr>
      <th data-sort-table="text">Cliente</th>
      <th data-sort-table="number">Total</th>
      <th data-sort-table="text">Estado</th>
    </tr>
  </thead>
  <tbody>
    <tr><td>Carlos</td><td>120000</td><td>Pagado</td></tr>
    <tr><td>Laura</td><td>80000</td><td>Pendiente</td></tr>
  </tbody>
</table>
```

CSS del componente

styles.css

```
.data-table {
  width: 100%;
  border-collapse: collapse;
  overflow: hidden;
  border-radius: 16px;
}
.data-table th,
.data-table td {
  padding: 12px;
  border-bottom: 1px solid #e2e8f0;
  text-align: left;
}
.data-table th {
  background: #0f172a;
  color: #fff;
  cursor: pointer;
  user-select: none;
}
.data-table tr:hover td { background: #f8fafc; }
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js

```
export class SortableTable {
  constructor(tableId) {
    this.table = document.getElementById(tableId);
    this.direction = 1;
    this.lastColumn = null;
    this.init();
  }

  init() {
    if (!this.table) return;
    this.table.querySelectorAll('[data-sort-table]').forEach((th, index) => {
      th.addEventListener('click', () => this.sortByColumn(index, th.dataset.sortTable));
    });
  }

  sortByColumn(index, type) {
    const tbody = this.table.querySelector('tbody');
    const rows = [...tbody.querySelectorAll('tr')];

    if (this.lastColumn === index) this.direction *= -1;
    else this.direction = 1;
    this.lastColumn = index;

    rows.sort((a, b) => {
      const valueA = a.children[index].textContent.trim();
      const valueB = b.children[index].textContent.trim();
      if (type === 'number') return (Number(valueA) - Number(valueB)) * this.direction;
      return valueA.localeCompare(valueB) * this.direction;
    });

    tbody.append(...rows);
  }
}

// script.js
import { SortableTable } from './funciones.js';
new SortableTable('ordersTable');
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como `script-normal.js` y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class SortableTable {
  constructor(tableId) {
    this.table = document.getElementById(tableId);
    this.direction = 1;
    this.lastColumn = null;
    this.init();
  }

  init() {
    if (!this.table) return;
    this.table.querySelectorAll('[data-sort-table]').forEach((th, index) => {
      th.addEventListener('click', () => this.sortByColumn(index, th.dataset.sortTable));
    });
  }

  sortByColumn(index, type) {
    const tbody = this.table.querySelector('tbody');
    const rows = [...tbody.querySelectorAll('tr')];
    if (this.lastColumn === index) this.direction *= -1;
    else this.direction = 1;
    this.lastColumn = index;
    rows.sort((a, b) => {
      const valueA = a.children[index].textContent.trim();
      const valueB = b.children[index].textContent.trim();
      if (type === 'number') return (Number(valueA) - Number(valueB)) * this.direction;
      return valueA.localeCompare(valueB) * this.direction;
    });
    tbody.append(...rows);
  }
}

new SortableTable('ordersTable');
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
ordersTable	Id de la tabla.	Cambia por el id real de tu tabla.
data-sort-table	Tipo de ordenamiento.	Usa text o number.
direction	Direccion asc/desc.	Se invierte al hacer clic dos veces en la misma columna.
children[index]	Celda correspondiente a la columna.	No debe haber celdas combinadas complicadas.

Implementacion paso a paso

- 1 Crea una tabla con thead y tbody.
- 2 Agrega data-sort-table a los th que quieras ordenar.
- 3 Copia la clase SortableTable.
- 4 Instancia con el id de la tabla.
- 5 Prueba orden en columnas de texto y numero.

Errores comunes y como solucionarlos

- Ordena mal numeros con simbolos: limpia valores antes de Number.
- No funciona con celdas colspan: evita estructuras complejas para tablas ordenables simples.
- No se nota que es clickeable: agrega cursor pointer y una flecha visual.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

38. Filtro de tabla por texto

Aspecto	Explicacion
Que hace	Filtra filas de una tabla segun lo que el usuario escribe.
Donde se usa en industria	Buscar usuarios, productos, facturas, pedidos, inventarios, registros o historiales.
Como se personaliza	Cambia el id del input, el id de la tabla y el criterio de busqueda.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<input type="search" id="tableFilter" placeholder="Filtrar tabla...">

<table class="data-table" id="ordersTable">
  <tbody>
    <tr><td>Carlos</td><td>120000</td><td>Pagado</td></tr>
    <tr><td>Laura</td><td>80000</td><td>Pendiente</td></tr>
  </tbody>
</table>
```

CSS del componente

styles.css

```
#tableFilter {
  width: 100%;
  padding: 12px 14px;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
  margin-bottom: 16px;
}
tr.is-hidden {
  display: none;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class TableFilter {
  constructor(inputId, tableId) {
    this.input = document.getElementById(inputId);
    this.table = document.getElementById(tableId);
    this.init();
  }

  init() {
    if (!this.input || !this.table) return;
    this.input.addEventListener('input', () => this.filter());
  }

  filter() {
    const term = this.input.value.trim().toLowerCase();
    const rows = this.table.querySelectorAll('tbody tr');

    rows.forEach((row) => {
      const text = row.textContent.toLowerCase();
      row.classList.toggle('is-hidden', !text.includes(term));
    });
  }
}

// script.js
import { TableFilter } from './funciones.js';
new TableFilter('tableFilter', 'ordersTable');
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class TableFilter {
  constructor(inputId, tableId) {
    this.input = document.getElementById(inputId);
    this.table = document.getElementById(tableId);
    this.init();
  }

  init() {
    if (!this.input || !this.table) return;
    this.input.addEventListener('input', () => this.filter());
  }

  filter() {
    const term = this.input.value.trim().toLowerCase();
    const rows = this.table.querySelectorAll('tbody tr');
    rows.forEach((row) => {
      const text = row.textContent.toLowerCase();
      row.classList.toggle('is-hidden', !text.includes(term));
    });
  }
}

new TableFilter('tableFilter', 'ordersTable');
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
tableFilter	Id del input de busqueda.	Cambia segun tu HTML.
ordersTable	Id de la tabla.	Debe existir en el documento.
row.textContent	Texto completo de cada fila.	Puedes limitar a columnas especificas si quieres.
.is-hidden	Clase para ocultar filas.	Puedes usar visibility si quieres conservar espacio, pero no es comun.

Implementacion paso a paso

- 1 Crea un input search arriba de la tabla.
- 2 Asegura que la tabla tenga tbody.

- 3 Copia TableFilter.
- 4 Instancia con id del input y tabla.
- 5 Prueba con texto de distintas columnas.

Errores comunes y como solucionarlos

- No filtra: la tabla no tiene tbody o el id no coincide.
- Filtra muy lento en tablas enormes: combina con debounce.
- No encuentra acentos: normaliza texto con normalize si lo necesitas.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

39. Notas con localStorage

Aspecto	Explicacion
Que hace	Guarda notas simples en el navegador sin base de datos.
Donde se usa en industria	Borradores, notas rapidas, preferencias, tareas simples, formularios temporales y prototipos.
Como se personaliza	Cambia la clave localStorage, campos de nota y forma de renderizar.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<form id="noteForm" class="form">
  <input id="noteTitle" placeholder="Titulo de la nota" required>
  <textarea id="noteText" placeholder="Escribe una nota" required></textarea>
  <button class="btn" type="submit">Guardar nota</button>
</form>

<section id="notesList" class="notes-list"></section>
```

CSS del componente

styles.css

```
.notes-list {
  display: grid;
  gap: 12px;
  margin-top: 18px;
}
.note {
  padding: 16px;
  border-radius: 16px;
  background: #fff;
  border: 1px solid #e2e8f0;
}
.note_actions {
  display: flex;
  justify-content: flex-end;
  gap: 8px;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export function readJSON(key, fallback = []) {
  try {
    return JSON.parse(localStorage.getItem(key)) ?? fallback;
  } catch {
    return fallback;
  }
}

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export class NotesApp {
  constructor({ formId, titleId, textId, listId, storageKey = 'kit-notes' }) {
    this.form = document.getElementById(formId);
    this.title = document.getElementById(titleId);
    this.text = document.getElementById(textId);
    this.list = document.getElementById(listId);
    this.storageKey = storageKey;
    this.notes = readJSON(this.storageKey, []);
    this.init();
  }

  init() {
    this.render();
    this.form?.addEventListener('submit', (event) => {
      event.preventDefault();
      this.addNote();
    });
    this.list?.addEventListener('click', (event) => {
      const deleteButton = event.target.closest('[data-delete-note]');
      if (deleteButton) this.deleteNote(deleteButton.dataset.deleteNote);
    });
  }

  addNote() {
    const note = {
      id: crypto.randomUUID(),
      title: this.title.value.trim(),
      text: this.text.value.trim(),
      createdAt: new Date().toISOString()
    };
    this.notes.unshift(note);
    saveJSON(this.storageKey, this.notes);
    this.form.reset();
    this.render();
  }

  deleteNote(id) {
    this.notes = this.notes.filter((note) => note.id !== id);
    saveJSON(this.storageKey, this.notes);
    this.render();
  }

  render() {
    if (!this.list) return;
    this.list.innerHTML = this.notes.map((note) => `
      <article class="note">
        <h3>${note.title}</h3>
        <p>${note.text}</p>
        <div class="note__actions">
          <button type="button" data-delete-note="${note.id}">Eliminar</button>
        </div>
      </article>
    `).join('');
  }
}

// script.js
import { NotesApp } from './funciones.js';
new NotesApp({ formId: 'noteForm', titleId: 'noteTitle', textId: 'noteText', listId: 'notesList' });

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
function readJSON(key, fallback = []) {
  try { return JSON.parse(localStorage.getItem(key)) || fallback; }
  catch { return fallback; }
}
function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}
class NotesApp {
  constructor({ formId, titleId, textId, listId, storageKey = 'kit-notes' }) {
    this.form = document.getElementById(formId);
    this.title = document.getElementById(titleId);
    this.text = document.getElementById(textId);
    this.list = document.getElementById(listId);
    this.storageKey = storageKey;
    this.notes = readJSON(this.storageKey, []);
    this.init();
  }
  init() {
    this.render();
    this.form.addEventListener('submit', (event) => { event.preventDefault(); this.addNote(); });
    this.list.addEventListener('click', (event) => {
      const deleteButton = event.target.closest('[data-delete-note]');
      if (deleteButton) this.deleteNote(deleteButton.dataset.deleteNote);
    });
  }
  addNote() {
    const note = { id: Date.now().toString(), title: this.title.value.trim(), text: this.text.value.trim(), createdAt: new Date().toISOString() };
    this.notes.unshift(note);
    saveJSON(this.storageKey, this.notes);
    this.form.reset();
    this.render();
  }
  deleteNote(id) {
    this.notes = this.notes.filter((note) => note.id !== id);
    saveJSON(this.storageKey, this.notes);
    this.render();
  }
  render() {
    this.list.innerHTML = this.notes.map((note) => `<article class="note"><h3>${note.title}</h3><p>${note.text}</p><button data-delete-note="${note.id}">Eliminar</button></article>`).join('');
  }
}
new NotesApp({ formId: 'noteForm', titleId: 'noteTitle', textId: 'noteText', listId: 'notesList' });
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
kit-notes	Clave de localStorage.	Cambiala para no mezclar proyectos.
noteTitle/noteText	Campos del formulario.	Puedes agregar categoria, prioridad o fecha.
crypto.randomUUID()	Genera id unico.	Usa Date.now si quieres compatibilidad simple.
readJSON/saveJSON	Helpers reutilizables.	Usalos para cualquier dato guardado.

Implementacion paso a paso

- 1 Crea formulario y contenedor de notas.
- 2 Copia helpers readJSON y saveJSON.
- 3 Copia NotesApp.
- 4 Inicializa con los ids correctos.
- 5 Crea, recarga la pagina y confirma que la nota sigue guardada.

Errores comunes y como solucionarlos

- No guarda: revisa que localStorage no este bloqueado.
- Se duplican notas: no inicialices NotesApp dos veces.

- Se borra todo al cambiar de proyecto: usa storageKey diferente por app.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con ToastManager para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en localStorage si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con aria-label, aria-hidden, focus-visible y navegacion por teclado cuando aplique.

40. Exportar JSON

Aspecto	Explicacion
Que hace	Descarga datos del navegador o de la app como archivo .json.
Donde se usa en industria	Exportar reportes, backups, notas, configuraciones, carritos, filtros o datos de prototipo.
Como se personaliza	Cambia el origen de datos, nombre del archivo y estructura del JSON.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<button class="btn" id="exportNotes">Exportar notas JSON</button>
```

CSS del componente

styles.css

```
.btn-export {  
  background: linear-gradient(135deg, #0f766e, #2563eb);  
  color: #fff;  
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class JSONExporter {
  static download(data, filename = 'datos.json') {
    const json = JSON.stringify(data, null, 2);
    const blob = new Blob([json], { type: 'application/json' });
    const url = URL.createObjectURL(blob);

    const link = document.createElement('a');
    link.href = url;
    link.download = filename;
    document.body.appendChild(link);
    link.click();
    link.remove();

    URL.revokeObjectURL(url);
  }
}

// script.js
import { JSONExporter, readJSON } from './funciones.js';
document.getElementById('exportNotes').addEventListener('click', () => {
  const notes = readJSON('kit-notes', []);
  JSONExporter.download(notes, 'mis-notas.json');
});
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class JSONExporter {
  static download(data, filename = 'datos.json') {
    const json = JSON.stringify(data, null, 2);
    const blob = new Blob([json], { type: 'application/json' });
    const url = URL.createObjectURL(blob);
    const link = document.createElement('a');
    link.href = url;
    link.download = filename;
    document.body.appendChild(link);
    link.click();
    link.remove();
    URL.revokeObjectURL(url);
  }
}

document.getElementById('exportNotes').addEventListener('click', () => {
  const notes = JSON.parse(localStorage.getItem('kit-notes')) || [];
  JSONExporter.download(notes, 'mis-notas.json');
});
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
data	Informacion que quieres exportar.	Puede venir de localStorage, un array o una respuesta de API.
filename	Nombre del archivo descargado.	Usa extension .json.
Blob	Objeto que representa archivo en memoria.	Cambia type si exportas otro formato.
URL.revokeObjectURL	Libera memoria despues de descargar.	Dejalo para buenas practicas.

Implementacion paso a paso

- 1 Crea un boton de exportacion.
- 2 Define de donde saldrán los datos: array, localStorage o API.
- 3 Copia JSONExporter.

- 4 Llama `JSONExporter.download(datos, nombreArchivo)`.
- 5 Abre el archivo descargado y revisa que tenga estructura correcta.

Errores comunes y como solucionarlos

- El archivo sale vacio: revisa que los datos existan antes de exportar.
- El JSON sale en una sola linea: usa `JSON.stringify(data, null, 2)`.
- El navegador bloquea la descarga: asegúrate de llamar download desde un clic del usuario.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con `ToastManager` para mostrar mensajes de exito o error.
- Agregar estados de carga si depende de datos externos.
- Guardar preferencias en `localStorage` si el usuario puede cambiar configuraciones.
- Mejorar accesibilidad con `aria-label`, `aria-hidden`, `focus-visible` y navegacion por teclado cuando aplique.

Cierre: como combinar las funcionalidades 26 a 40

Estas funcionalidades se pueden usar de manera individual, pero en proyectos reales normalmente se combinan. Por ejemplo, un catalogo profesional puede usar Cards dinamicas para mostrar productos, Busqueda debounce para filtrar por texto, Filtros chips para categorias, Ordenamiento para precio o nombre, Paginacion para dividir resultados, Modal para ver detalle del producto, Toast para confirmar acciones y Copy clipboard para copiar un codigo de descuento.

Patron recomendado para combinar busqueda + filtros + orden + paginacion

Ejemplo de estado global

```
const state = {
  originalItems: products,
  searchTerm: '',
  category: 'all',
  sort: 'name-asc',
  page: 1,
  perPage: 6,
};

function applyState() {
  let results = [...state.originalItems];

  if (state.searchTerm !== '') {
    results = results.filter((item) => item.name.toLowerCase().includes(state.searchTerm));
  }

  if (state.category !== 'all') {
    results = results.filter((item) => item.category === state.category);
  }

  results.sort((a, b) => a.name.localeCompare(b.name));

  const start = (state.page - 1) * state.perPage;
  const pageItems = results.slice(start, start + state.perPage);

  catalog.render(pageItems);
}
```

Checklist final general

- Si usas modulos, todas las clases reutilizables deben exportarse desde funciones.js.
- Si usas version normal, no uses import/export.
- Mantén nombres claros para ids y data-attributes.
- Usa localStorage solo para prototipos, preferencias o datos no sensibles.
- No guardes contraseñas, tokens privados o informacion delicada en localStorage.
- Documenta que variables cambiaste para poder reutilizar el componente despues.

Actualizacion puntual 2026 - Funcionalidades 26 a 40

~~88. Originalmente, el sistema de gestión de recursos humanos (SGRH) no permitía la actualización puntual.~~